WEEK 3 · DAY 2 · RULE BOOK

Control Charts and Residuals

Follow it step by step. Every common error and its fix is here.

Your goal today: build a control chart for the solar array, make it smarter (rolling + robust), then catch surprises in the amplifier with residuals. Earn **14 green ✅ ticks**. Time: about 60 minutes. Keep this page open next to Colab.

Before you start (2 minutes)

- ☐ Open `W3D2-exercise` in Colab, then **File → Save a copy in Drive**. Work only in the copy.
- ☐ Run **STEP 0** and wait for ✅ `Ready!`
- ☐ Run the cells **top to bottom**. Part B uses `bus` and `tw` from earlier cells.

The 7 golden rules of today

#	Rule	Why
1	**Limits come from a quiet reference period**, never from the whole file	The whole file contains the faults you're trying to catch
2	**Control limits = centre ± 3σ** (not min/max, not design limits)	3σ keeps false alarms to ~8 per 2,880 normal hours
3	**One set of limits per operating state** (eclipse, recharge, sunlit)	Mixing states makes σ 8× bigger and the chart goes blind
4	**Trailing windows only**, with `.shift(1)`	A reading can't help judge itself; centred windows peek at the future
5	**Robust = median + MAD × 1.4826**	One wild hour can't poison the ruler
6	**Residual = observed − expected**, in that order	Positive = hotter / higher than expected
7	**Check the residual is clean** before you trust it	Mean ≈ 0, no autocorrelation left, constant spread

The exercise, step by step

Task	What you do	The tool you need
A1	Count recharge hours	`(bus["state"] == "recharge").sum()`
A2	Centre and σ of the reference fortnight	`ref.mean()` , `ref.std()`

Task	What you do	The tool you need
A3	Lower and upper control limits	`centre - 3 * sigma`, `centre + 3 * sigma`
A4	How many sunlit hours fall outside the August band?	`outside.sum()`
A5	Chance false alarms at 3σ and 2σ	`hours * 0.0027`, `hours * 0.0455`
B1	Count rolling robust flags	`(z.abs() > 3).sum()`
B2	Two letters: why median/MAD, why 1.4826	read the options
B3	Residual of the amplifier	`tw["twta_temp_c"] - expected`
B4	Is the residual clean? `"yes"` / `"no"`	read the 3 health numbers
B5	The troublemaker day	`residual[train].abs().idxmax()`
B6	How many σ is the spike?	`residual2.loc["2024-09-03"] / sigma2`
B7	Days flagged after 12 May 2025	`(z2.loc["2025-05-12":].abs() > 3).sum()`
B8	Rolling detector flags after 12 May	`(rz.loc["2025-05-12":].abs() > 3).sum()`
🌶️ **Bonus**	Does tampering make it worse?	`tampered.std() / calm.std()`

The control-chart recipe

```
ref = series.loc["START":"END"]          # a quiet, known-good stretch, ONE operating
state
centre, sigma = ref.mean(), ref.std()
lcl, ucl = centre - 3 * sigma, centre + 3 * sigma
flags = (series < lcl) | (series > ucl)
```

The rolling robust recipe

```
med = x.rolling(168, min_periods=48).median().shift(1)                    # typical
value, from the past only
robust_sigma = (x - med).abs().rolling(168, min_periods=48).median().shift(1) * 1.4826
z = (x - med) / robust_sigma
flags = z.abs() > 3
```

When things go wrong: error → fix

You see	It means	Fix
`AssertionError: ✍️ Do A2 and A3 first`	`lcl` / `ucl` are still blank	Finish A2 and A3, then run the plot cell again
`TypeError: unsupported operand ... NoneType`	You used a variable that is still `___`	Fill in the blank above it
`KeyError: '2024-09-03'`	The date isn't in the index, or the wrong table	Use `residual2.loc["2024-09-03"]` (a Series with dates as index)
B3 says ❌ but your number looks right	You gave one number, not the whole column	`residual = tw["twta_temp_c"] - expected`
A4 is huge (1,000+)	That's correct!	Read the message: the August band went stale
B8 says 0 and you think it's wrong	It's right	The rolling window swallowed the shift. That's the lesson
A chart is empty	You ran it before the cells above	Runtime → Run before, then run it again

Stuck? Climb the ladder

1. **Read the ❌ hint**: it says what's wrong.
2. **Check this rule book's recipes.**
3. **Ask your neighbour.**
4. **Ask the AI button** to *explain the error* (course data is synthetic, so it's safe here).
5. **Raise your hand.**

🏁 **Finished?** Run `scoreboard()`. 14 of 14 = 🏆. Then try the 🌶️ bonus and explain to your neighbour why tampering hurts.